

- [Modélisation Mathématique de DataPipe](#)
 - [1. Introduction](#)
 - [2. Notations Générales](#)
 - [3. Pipeline Comme Graphe Orienté Acyclique](#)
 - [4. Fonction de Transformation d'un Noeud](#)
 - [5. Opérateurs de Transformation](#)
 - [5.1 Filtrage](#)
 - [5.2 Projection et Mapping](#)
 - [5.3 Agrégation](#)
 - [5.4 Tri](#)
 - [5.5 Dédoublonnage](#)
 - [5.6 Jointure](#)
 - [5.7 Transformation SQL](#)
 - [5.8 Validation de Données](#)
 - [6. Détection d'Anomalies](#)
 - [7. Transformation Assistée par IA](#)
 - [8. Modèle d'Exécution](#)
 - [9. Métriques de Performance](#)
 - [10. Qualité des Données](#)
 - [10.1 Complétude](#)
 - [10.2 Unicité](#)
 - [10.3 Taux d'Anomalies](#)
 - [10.4 Taux de Validité](#)
 - [11. Exemple Bancaire : Détection d'Anomalies](#)
 - [12. Limites du Modèle Actuel](#)
 - [13. Évolutions Recommandées](#)
 - [14. Conclusion](#)

Modélisation Mathématique de DataPipe

1. Introduction

DataPipe modélise un pipeline ETL bancaire comme un système de transformation de données structuré autour d'un graphe orienté acyclique. Chaque pipeline reçoit une ou plusieurs sources de données, applique une suite ordonnée de transformations, puis

produit des sorties exploitables pour l'analyse, le reporting, l'audit ou l'automatisation bancaire.

L'objectif de cette modélisation est de formaliser le comportement du système afin de garantir la cohérence de l'exécution, la traçabilité des transformations, la fiabilité des résultats, la détection d'anomalies et l'évaluation de la performance.

2. Notations Générales

On note :

- P : un pipeline DataPipe ;
- $G = (V, E)$: le graphe associé au pipeline ;
- $V = \{v_1, v_2, \dots, v_n\}$: l'ensemble des noeuds du pipeline ;
- $E \subseteq V \times V$: l'ensemble des arêtes entre noeuds ;
- D_i : le dataset produit par le noeud v_i ;
- T_i : la transformation appliquée par le noeud v_i ;
- C_i : la configuration du noeud v_i ;
- $A = \{a_1, a_2, \dots, a_m\}$: l'ensemble des attributs ou colonnes.

Un dataset est représenté comme une relation tabulaire :

$$D = \{r_1, r_2, \dots, r_k\}$$

Chaque ligne r_j est un vecteur de valeurs :

$$r_j = (x_{j1}, x_{j2}, \dots, x_{jm})$$

3. Pipeline Comme Graphe Orienté Acyclique

Un pipeline DataPipe est représenté par un graphe orienté :

$$G = (V, E)$$

Un noeud $v_i \in V$ correspond à une opération élémentaire : lecture de fichier, filtrage, agrégation, jointure, transformation SQL, validation, export ou transformation assistée par IA.

Une arête $e = (v_i, v_j) \in E$ signifie que la sortie du noeud v_i devient une entrée du noeud v_j .

Pour qu'un pipeline soit exécutable, le graphe doit être acyclique :

$$\nexists (v_1, v_2, \dots, v_k) \text{ tel que } v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k \rightarrow v_1$$

Cette contrainte garantit qu'il existe un ordre d'exécution valide. DataPipe applique un tri topologique :

$$\text{order}(G) = (v_{\pi(1)}, v_{\pi(2)}, \dots, v_{\pi(n)})$$

tel que pour toute arête $(v_i, v_j) \in E$, le noeud v_i est exécuté avant v_j :

$$(v_i, v_j) \in E \Rightarrow \pi^{-1}(i) < \pi^{-1}(j)$$

4. Fonction de Transformation d'un Noeud

Chaque noeud est modélisé comme une fonction :

$$T_i : D^{p_i} \times C_i \rightarrow D$$

où :

- D est l'espace des datasets ;
- p_i est le nombre d'entrées du noeud ;
- C_i est la configuration du noeud ;
- T_i produit un dataset de sortie.

La sortie d'un noeud est donc :

$$D_i = T_i(D_{\text{parent}_1}, D_{\text{parent}_2}, \dots, D_{\text{parent}_p}, C_i)$$

Pour les noeuds sources, comme `csv_reader` ou `json_reader`, la transformation ne dépend pas d'un parent :

$$D_i = T_i(C_i)$$

5. Opérateurs de Transformation

5.1 Filtrage

Le filtrage conserve uniquement les lignes qui satisfont une condition logique φ .

$$T_{filter}(D, \varphi) = \{r \in D \mid \varphi(r) = vrai\}$$

Si plusieurs conditions sont définies avec une logique **AND**, on obtient :

$$\varphi(r) = \varphi_1(r) \wedge \varphi_2(r) \wedge \dots \wedge \varphi_k(r)$$

Avec une logique **OR**, on obtient :

$$\varphi(r) = \varphi_1(r) \vee \varphi_2(r) \vee \dots \vee \varphi_k(r)$$

5.2 Projection et Mapping

Le mapping sélectionne, renomme ou réorganise les attributs d'un dataset. Pour un ensemble de colonnes $B \subseteq A$, la projection est :

$$T_{map}(D, B) = \pi_B(D)$$

Chaque ligne de sortie contient uniquement les attributs sélectionnés ou renommés.

5.3 Agrégation

L'agrégation regroupe les lignes selon un ensemble de clés K , puis applique des fonctions statistiques F .

$$T_{agg}(D, K, F) = \gamma_{K,F}(D)$$

Pour un groupe g , la sortie est :

$$r_g = (K_g, F_1(D_g), F_2(D_g), \dots, F_q(D_g))$$

Les fonctions prises en charge incluent :

- comptage : $COUNT(D_g) = |D_g|$;
- somme : $SUM(x) = \sum_{i=1}^n x_i$;
- moyenne : $AVG(x) = \frac{1}{n} \sum_{i=1}^n x_i$;
- minimum : $MIN(x)$;
- maximum : $MAX(x)$;

- nombre de valeurs distinctes : $COUNT_DISTINCT(x)$.

5.4 Tri

Le tri ordonne les lignes selon une ou plusieurs clés.

$$T_{sort}(D, a, ordre) = \text{sort}(D, a, ordre)$$

avec :

$$ordre \in \{asc, desc\}$$

5.5 Dédoublonnage

Le dédoublonnage conserve une seule occurrence pour une clé donnée K .

$$T_{dedup}(D, K) = \{r \in D \mid key(r) \notin S\}$$

où S est l'ensemble des clés déjà rencontrées. Selon la configuration, DataPipe conserve la première ou la dernière occurrence.

5.6 Jointure

La jointure combine deux datasets D_L et D_R selon deux clés k_L et k_R .

Pour une jointure interne :

$$T_{join}(D_L, D_R) = \{r_L \cup r_R \mid r_L \in D_L, r_R \in D_R, r_L[k_L] = r_R[k_R]\}$$

DataPipe prend aussi en charge les variantes **left**, **right** et **full**, qui conservent respectivement les lignes non appariées du dataset gauche, droit ou des deux.

5.7 Transformation SQL

Une transformation SQL est modélisée comme une fonction déclarative :

$$T_{sql}(D, q) = q(D)$$

où q est une requête **SELECT** appliquée au dataset d'entrée représenté comme une table temporaire **input**.

Pour des raisons de sécurité, seules les transformations non destructives sont autorisées. Les opérations comme **DROP**, **DELETE**, **ALTER**, **INSERT** ou **UPDATE** sont rejetées.

5.8 Validation de Données

La validation applique un ensemble de règles R à chaque ligne.

$$T_{\text{validate}}(D, R) = \{r \in D \mid \forall \rho \in R, \rho(r) = \text{vrai}\}$$

Une règle typique est la contrainte de présence :

$$\rho_{\text{required}}(r, a) = \text{vrai} \iff r[a] = \text{null} \wedge r[a] = ""$$

Les lignes invalides sont écartées et journalisées.

6. Détection d'Anomalies

Dans le contexte bancaire, une anomalie correspond à une transaction dont le comportement s'écarte fortement du comportement statistique attendu.

Pour une variable numérique X , par exemple le montant d'une transaction, on calcule la moyenne :

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i$$

puis l'écart type :

$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2}$$

Le score d'anomalie utilisé est le z-score :

$$z_i = \frac{|x_i - \mu|}{\sigma}$$

Une ligne est considérée comme anomalie si :

$$z_i > \tau$$

avec :

$$\tau = 3$$

Interprétation :

- $z_i \leq 3$: transaction considérée comme normale ;
- $3 < z_i \leq 5$: anomalie moyenne ;
- $z_i > 5$: anomalie forte.

Cette approche est simple, explicable et adaptée à une première analyse de transactions. Elle est particulièrement utile pour détecter des montants extrêmes, mais elle peut être complétée par des méthodes plus avancées lorsque les données deviennent volumineuses ou multidimensionnelles.

7. Transformation Assistée par IA

La transformation IA est modélisée comme une fonction de génération :

$$G_{IA} : (d, A) \rightarrow q$$

où :

- d est une description en langage naturel ;
- A est la liste des colonnes disponibles ;
- q est une requête SQL générée.

La transformation complète devient :

$$T_{IA}(D, d) = T_{sql}(D, G_{IA}(d, A))$$

Exemple de description utilisateur :

Calculer la somme des transactions par mois.

Requête générée :

```
SELECT STRFTIME('%Y-%m', date_transaction) AS mois,  
       SUM(montant) AS total  
FROM input
```

```
GROUP BY mois  
ORDER BY mois DESC;
```

Le rôle de l'IA est donc de convertir une intention métier en transformation formelle exécutable.

8. Modèle d'Exécution

L'exécution d'un pipeline suit trois étapes :

1. Validation structurelle du graphe.
2. Tri topologique des noeuds.
3. Exécution séquentielle des transformations selon l'ordre obtenu.

Pour chaque noeud v_i , DataPipe calcule :

$$D_i = T_i(inputs(v_i), C_i)$$

avec :

$$inputs(v_i) = \{D_j \mid (v_j, v_i) \in E\}$$

Chaque exécution produit des métriques opérationnelles :

- nombre de lignes reçues ;
- nombre de lignes produites ;
- durée d'exécution ;
- colonnes de sortie ;
- aperçu du résultat ;
- logs d'information, d'avertissement ou d'erreur.

9. Métriques de Performance

Pour un pipeline P , la durée totale d'exécution est :

$$T(P) = \sum_{i=1}^n T(v_i)$$

où $T(v_i)$ est la durée d'exécution du noeud v_i .

Le débit peut être estimé par :

$$Q(P) = \frac{N_{out}}{T(P)}$$

où N_{out} est le nombre de lignes produites par le pipeline.

Le taux de réduction des données est :

$$\rho = 1 - \frac{N_{out}}{N_{in}}$$

avec :

- N_{in} : nombre total de lignes en entrée ;
- N_{out} : nombre total de lignes en sortie.

Un taux élevé peut indiquer un filtrage important, une validation stricte ou une agrégation forte.

10. Qualité des Données

La qualité d'un dataset peut être évaluée à l'aide de plusieurs indicateurs.

10.1 Complétude

$$Completeness(D) = 1 - \frac{N_{missing}}{N_{total}}$$

10.2 Unicité

$$Uniqueness(D, K) = \frac{|distinct(K)|}{|D|}$$

10.3 Taux d'Anomalies

$$AnomalyRate(D) = \frac{N_{anomalies}}{|D|}$$

10.4 Taux de Validité

$$Validity(D) = \frac{N_{valid}}{|D|}$$

Ces métriques permettent d'évaluer la fiabilité des données avant leur export ou leur exploitation métier.

11. Exemple Bancaire : Détection d'Anomalies

Soit un dataset de transactions :

$$D = \{(id, montant, type, statut, date)\}$$

On souhaite détecter les transactions anormalement élevées.

Pipeline :

CSV Reader -> Validate -> Filter -> AI/SQL Transform -> Export

Formalisation :

$$D_1 = T_{csv}(fichier)$$

$$D_2 = T_{validate}(D_1, R)$$

$$D_3 = T_{filter}(D_2, statut = "traite")$$

$$D_4 = \{r \in D_3 \mid z(r.montant) > 3\}$$

$$D_5 = T_{export}(D_4)$$

Le résultat final contient les transactions les plus suspectes selon leur écart statistique à la moyenne.

12. Limites du Modèle Actuel

Le modèle actuel est robuste pour des pipelines ETL explicables, mais il présente certaines limites :

- le z-score est sensible aux valeurs extrêmes lorsqu'elles influencent fortement la moyenne ;
- les transformations sont exécutées séquentiellement, sans parallélisation globale du graphe ;
- la détection d'anomalies actuelle est principalement univariée ;
- la qualité des résultats IA dépend de la précision de la description utilisateur et des colonnes disponibles ;
- SQLite en mémoire est adapté aux transformations légères à moyennes, mais pas aux volumes massifs.

13. Évolutions Recommandées

Pour renforcer le modèle mathématique et analytique, les évolutions suivantes sont pertinentes.

Introduire des méthodes robustes comme le score MAD :

$$MAD = median(|x_i - median(X)|)$$

Utiliser des modèles multivariés :

$$score(x_i) = f(x_i, x_{i1}, x_{i2}, \dots, x_{im})$$

Ces évolutions permettraient d'améliorer la détection dans les cas où l'anomalie dépend de plusieurs variables simultanément : montant, heure, pays, type d'opération, historique client ou canal de paiement.

Autres pistes d'amélioration :

- ajouter une estimation de complexité par type de noeud ;
- paralléliser les branches indépendantes du DAG ;
- intégrer des contraintes de schéma fortes avant exécution ;
- calculer automatiquement les métriques de qualité après chaque transformation ;
- versionner les hypothèses statistiques utilisées par les pipelines critiques.

14. Conclusion

La modélisation mathématique de DataPipe repose sur une représentation claire : un pipeline est un graphe orienté acyclique dont les noeuds sont des fonctions de transformation appliquées à des relations tabulaires.

Cette approche combine les fondements des bases de données relationnelles, de l'algorithmique des graphes, de la statistique descriptive et de l'intelligence artificielle pour répondre à des besoins concrets de traitement de données bancaires.

Le modèle est explicable, extensible et aligné avec les exigences d'un environnement financier : traçabilité, contrôle, qualité des données et détection précoce des anomalies.